Lecture 28.02: Decision Tree Classifier — Recursive Binary Splitting & Hunt's Algorithm

📌 Executive Summary

The mechanics of recursive binary splitting, Hunt's greedy algorithm, evaluating continuous feature thresholds, and computing majority plurality voting at leaf nodes.

🏛️ Theoretical Formulation & Mathematical Architecture

⚙️ Hunt's Greedy Recursive Binary Splitting

Decision trees are constructed via **top-down greedy induction**. The general procedure, formalized in **Hunt's Algorithm** and implemented in modern CART, follows this recursive structure:

Let $\mathcal{D}_t$ be the set of training records that reach node t:

1. **Purity Base Case**: If all records in $\mathcal{D}_t$ belong to the same class y_k, node t is designated a **Leaf Node** predicting class y_k.
2. **Greedy Partitioning Step**: If $\mathcal{D}_t$ contains multiple classes, iterate through all d available features $j \in \{1, \dots, d\}$ and all valid candidate thresholds θ.
3. Evaluate the split (j, θ) that maximizes the **Impurity Reduction**:

$$\Delta I(t) = I(t) - \left(\frac{|\mathcal{D}_L|}{|\mathcal{D}_t|} I(\mathcal{D}_L) + \frac{|\mathcal{D}_R|}{|\mathcal{D}_t|} I(\mathcal{D}_R) \right)$$

4. Create child nodes $\mathcal{D}_L = \{\mathbf{x} \in \mathcal{D}_t \mid x_j \le \theta\}$ and $\mathcal{D}_R = \{\mathbf{x} \in \mathcal{D}_t \mid x_j > \theta\}$ and recursively repeat.

📐 Continuous Threshold Formulation

For a continuous feature X_j, sort the unique observed values in ascending order:

$$x_{(1),j} < x_{(2),j} < \cdots < x_{(u),j}$$

The set of valid split candidates Θ_j consists of the midpoints between adjacent sorted values:

$$\boxed{\theta_i = \frac{x_{(i),j} + x_{(i+1),j}}{2} \quad \text{for } i = 1, 2, \ldots, u-1}$$

📦 Majority Plurality Leaf Assignment

For any leaf node m containing a subset of samples $\mathcal{D}_m$, the empirical class probability for class k is:

$$\hat{p}_{mk} = \frac{1}{|\mathcal{D}_m|} \sum_{i \in \mathcal{D}_m} \mathbb{I}(y_i = k)$$

The deterministic class prediction $\hat{y}_m$ is the plurality winner:

$$\boxed{\hat{y}_m = \arg\max_k \hat{p}_{mk}}$$

💻 Vectorized Python Implementation

```
import numpy as np

def generate_candidate_thresholds(feature_values):
    unique_vals = np.sort(np.unique(feature_values))
    midpoints = (unique_vals[:-1] + unique_vals[1:]) / 2.0
    return midpoints

# Test
sample_feature = np.array([12.5, 5.0, 8.0, 5.0, 15.0])
candidates = generate_candidate_thresholds(sample_feature)
print("Unique Sorted:", np.unique(sample_feature))
print("Split Candidates:", candidates)
assert len(candidates) == len(np.unique(sample_feature)) - 1
```

🔬 Key Engineering Takeaways & Invariants

1. **Axis-Aligned Partitioning**: Decision trees recursively split feature space along orthogonal boundaries without requiring feature scaling.
2. **Impurity Optimization**: Split selection deterministically maximizes impurity reduction (Information Gain for classification, Variance Reduction for regression).
3. **Generalization Safeguards**: Regularization via pre-pruning (`max_depth` , `min_samples_leaf`) and post-pruning (`ccp_alpha`) prevents noisy leaf memorization.